

网络安全课程设计报告

(2022--2021年度第一学期)

课程名称：网络安全与管理
题　　目：CA系统
院　　系：计算机与信息技术学院
班　　级：19级大数据二班
组　　长：贾晓宸
指导教师：王春年
设计周数：4周

日期：2022年6月9日

组内成员	姓名	学号
组长	贾晓宸	201901004216

目录

1. **项目基本信息**
    - 项目名称
    - 实验目的
    - 背景与意义
    - 国内外应用现状
2. **需求分析**
    - 开发环境
    - 功能要求
    - 证书原理
    - 证书作用
    - 所采取技术
3. **实验设计**
    - 系统设计框架
    - 注册登录
    - 用户申请私人公钥
    - 接收用户数字证书的申请
    - 生成数字证书
    - 数据库存储证书
    - 下载证书
    - 接收用户的数字证书查询和注销
    - 产生和发布证书的有效期
4. **实验步骤**
    - 注册与登录
    - 申请公钥
    - 申请证书
    - 注销证书
    - 下载证书
    - 查询证书
5. **总结**
6. **附录**

一、项目基本信息

1. **项目名称**

   CA系统

2. **实验目的**

   实现一个ca系统，可以接受用户的认证请求，安全储存用户信息，记录储存对用户的一些认证信息，给用户颁发证书，可以吊销。

3. **背景与意义**

   CA认证系统，即CA证书颁发系统，是公钥基础设施（PKI）中的核心环节，是公钥加密过程中的第三方权威认证方，负责密钥和证书的产生、发布、管理、存储和撤销等功能，广泛用于电子商务等需要非对称加密的信息传输场景中。所有通过CA的信息传输方，都要无条件的信任CA的公正性，在消息传输的过程中，CA为信息传输的双方提供公私钥加密环境，提供身份认证、安全传输、不可否认性和数据完整性等功能。

   CA中心为每个使用公开密钥的用户发放一个数字证书，数字证书的作用是证明证书中列出的用户合法拥有证书中列出的公开密钥。CA机构的数字签名使得攻击者不能伪造和篡改证书。它负责产生、分配并管理所有参与网上交易的个体所需的数字证书，因此是安全电子交易的核心环节。在SET交易中，CA不仅对持卡人、商户发放证书，还要对获款的银行、网关发放证书。

   CA是证书的签发机构，它是公钥基础设施的核心。CA是负责签发证书、认证证书、管理已颁发证书的机关。它要制定政策和具体步骤来验证、识别用户身份，并对用户证书进行签名，以确保证书持有者的身份和公钥的拥有权。

   CA也拥有用户的证书（内含公钥）和私钥。网上的公众用户通过验证CA的签名从而信任CA，任何人都可以得到CA的证书（含公钥），用以验证CA所签发的证书。

   用户若欲获取证书，应先向CA提出申请，CA判明申请者的身份后，为之分配一个公钥，并将该公钥与其身份信息绑定，为该整体签名，签名后的整体即为证书，发还给申请者。如果一个用户想鉴别另一个证书的真伪，他就用CA的公钥对那个证书上的签字进行验证，一旦验证通过，该证书就被认为是有效的。

   为保证用户之间在网上传递信息的安全性、真实性、可靠性、完整性和不可抵赖性，不仅需要对用户的身份真实性进行验证，也需要有一个具有权威性、公正性、唯一性的机构，负责向电子商务的各个主体颁发并管理符合国际安全电子交易协议标准的电子商务安全证，并负责管理所有参与网上交易的个体所需的数字证书，因此CA是安全电子交易的核心环节。

   证书的内容包括：电子签证机关的信息、公钥用户信息、公钥、权威机构的签字和有效期等等。证书的格式和验证方法普遍遵循X.509 国际标准。

   公钥基础设施PKI(Publie Key Infrastrueture, 简称PKI)是利用公钥理论和技术建立的提供安全服务的基础设施, 是信息安全 的核心,也是电子商务安全的关键所在。PKI技术采用证书管理公钥, 通过第三方的可信任机构—认证中心CA(Certificate Authority),把用户的公钥和用户的其他标识信息(如名称、E-mail、身份证号等) 捆绑在一起,在Internet上验证用户的身份(其中认证机构CA是PKI系统的核心部分)。目前, 通用的办法是采用建立在PKI基础之上的数字证书,通过把要传输的数字信息进行加密和签名，保证信息传输的机密性、真实性、完整性和不可否认性,从而保证信息的安全传输。

4. **国内外应用现状**

   美国是最早提出PKI概念的国家，并于1996年成立了美国联邦PKI筹委会。与PKI相关的绝大部分标准都有美国制定，其PKI技术在世界上处于领先地位。2000年6月30日，美国总统克林顿正式签署美国《全球及全国商业电子签名法》，给与电子签名、数字证书以法律上的保护，这一决定使电子认证问题迅速成为各国政府关注的热点。加拿大在1993年就已经开始了政府PKI体系雏形的研究工作，到2000年已经在PKI体系方面获得重要进展。加拿大与美国代表了发达国家PKI发展的主流。

欧洲在PKI基础建设方面也成绩显著。已颁布了93/1999EC法规，强调技术中立、隐私权保护、国内与国外相互认证以及无歧视等原则。为了解决各国PKI之间的协同工作问题，他采取了一系列策略：如积极自主相关研究所、大学和企业研究PKI相关技术；自主PKI互操作性相关技术研究，并建立CA网络及其顶级CA。并于2000年10成立了欧洲桥CA指导委员会，于2001年3月23日成立欧洲桥CA。

我国的PKI技术从1998年开始起步，由于政府和各有关部门近年来对PKI产业的发展给予了高度重视，2001年PKI技术被列为"十五"863计划信息安全主题重大项目，并于同年10月成立了国家863计划信息安全基础设施研究中心。国家计委也在制定新的计划来支持PKI产业的发展，在国家电子政务工程中明确提出了要构建PKI体系。目前，我国已全面推动PKI技术研究于应用。

1998年国内第一家以实体形式运营的上海CA中心（SHECA）成立。目前，国内的CA机构分为区域型、行业型、商业性和企业型四类；截至2002年底，前三种CA机构已有60余家，58%的省市建立了区域CA，部分部委建立了行业CA。其中全国型的行业CA中心有中国金融认证中心CFCA、CTCA中国电信认证中心等。区域型CA有一定地区性，也称地区CA，如上海CA中心、广东电子商务认证中心。

二、需求分析

1. **开发环境**
   - 运行系统：windows10
   - 开发环境：IDEA2021.1, DataGrip2021.1.3, Java17, jQuery1.4.3, Mysql5.6.50
2. **功能要求**
   - 接受用户的提交申请，提交时候让用户自己产生公钥对;
   - 接受用户的申请，包括用户信息的表单提交，公钥的提交;
   - 在对用户实施认证的过程中，储存相应的电子文档，比如证书、营业执照的扫描文档;
   - 通过验证的给予颁发证书;
   - 用户密钥丢失时，可以吊销证书，密钥作废。
3. **证书原理**
   - 证书实际是由证书签证机关（CA）签发的对用户的公钥的认证。
     - 加密：CA认证将文字转换成不能直接阅读的形式（即密文）的过程称为加密。
     - 解密：将密文转换成能够直接阅读的文字（即明文）的过程称为解密
   - 如打算在电子文档上实现签名的目的，可使用数字签名。RSA公钥体制可实现对数字信息的数字签名，方法如下： * 信息发送者用其私钥对从所传报文中提取出的特征数据（或称数字指纹）进行RSA算法操作，以保证发信人无法抵赖曾发过该信息（即不可抵赖性），同时也确保信息报文在传递过程中未被篡改（即完整性）。当信息接收者收到报文后，就可以用发送者的公钥对数字签名进行验证。
     - 在数字签名中有重要作用的数字指纹是通过一类特殊的散列函数（HASH函数）生成的。对这些HASH函数的特殊要求是：
       a. 接受的输入报文数据没有长度限制
       b. 对任何输入报文数据生成固定长度的摘要（数字指纹）输出；
       c. 从报文能方便地算出摘要；
       d. 难以对指定的摘要生成一个报文，而由该报文可以算出该指定的摘要；
       e. 两个不同的报文难以生成具有相同的摘要。
4. **证书作用**
   - 证书主要有两个作用，一个是加密通信，一个是数字签名。

     * 加密通信是保证数据不被别人截获并且不被知道通信内容的，主要是两个层次，一个是通信双方身份确认，避免对方是冒充的，另一个是数据通过公钥加密传输和使用私钥解密。这方面常见的具体应用就是SSL和HTTPS，比如说WebSphere的管理员登陆和一些重要的登陆操作。 * 数字签名是用于识别签名者身份的，这个从字面就可以理解。你使用你的私钥进行签名，然后用户看到你的签名后用公钥检查，发现的确是你的数字签名，就可以了。这个常见的应用有代码发行商签名，就是签名控件的那种，邮件数字签名，电子公章等。

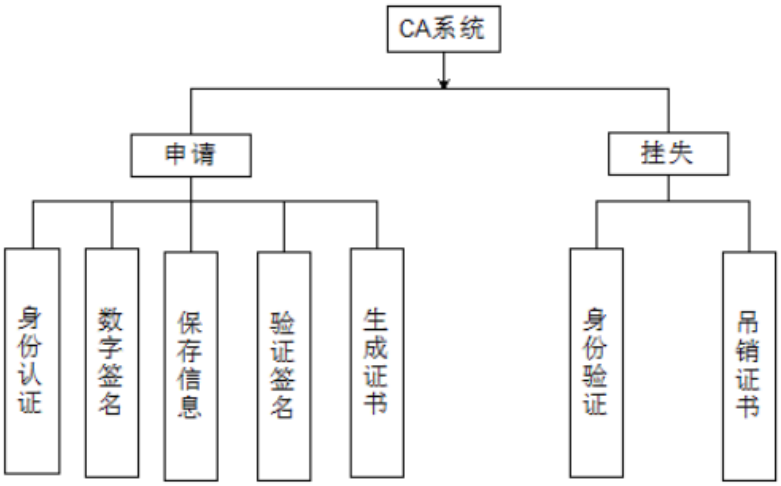
5. **所采取技术**

   - HTML+CSS+JavaScript
   - JDBC+MySQL
   - jQuery
   - md5加密方式
   - servlet+jsp

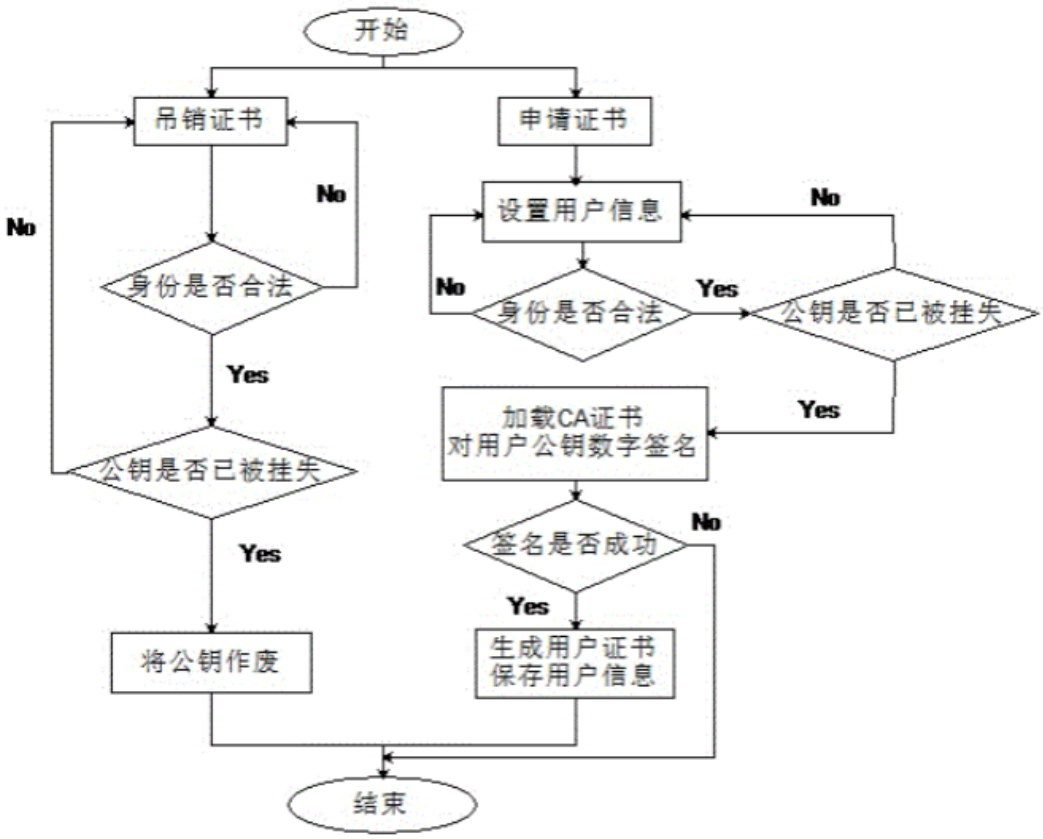
三、实验设计

1. **系统设计框架**

   首先，用户先申请自己的私人公钥，用于CA证书申请时使用。然后开始监听用户的信息传送，当用户通过验证并把用户信息和用户的公钥进行签名，然后对用户的信息进行封装加密，生成数字证书，并保留用户信息，最后就是验证数字签名。如果用户公钥丢失，用户需要验证自己的身份，通过验证才能进行挂失；如果公钥已经挂失，则挂失失败。



本次课程设计涉及到主界面的设计，具体的系统包括内部的程序设计以及运行界面的设计，内部的程序主要是将界面输入的数据接收，连接数据库进行用户信息的验证，以及证书的吊销；然后在后台进行用户信息的处理，包括数字签名以及封装数字证书，最后把扫描数字签名的结果放主界面显示，运行流程图如下：



2. **注册登录**

   - 注册
     - 用户注册需要输入用户名，密码，电子邮箱，后端自动生成创建时间与更新时间，将其数据存储到数据库
     - 用户名必须为字母+数字组合，长度5~16位
     - 密码包括数字+字母，长度5~16位，经过md5加密后存储到数据库users表中
     - 电子邮箱符合国际通用格式
   - 登录
     - 用户登录需输入用户名与密码，用户名与密码必须为同一用户，前端通过JavaScript进行密码md5加密，传输到后端，后端从数据库中提取相对应的用户信息，进行与用户输入的密码对比，如果匹配成功，则验证登录成功。

3. **用户申请私人密钥**

   用RSA算法为用户生成公钥pk.key和私钥sk.key,两者共同组成一对1024bits的RSA公私密钥对。

4. **接收用户的数字证书申请**

   申请证书页面包括一个container，其中有用户申请证书需要提交的表单。该表单包括：

   - 申请证书的用户名
   - 公钥（申请公钥时下载exe文件后自动生成的pk.key文件内容）
   - 国家
   - 州市
   - 地区
   - 组织
   - 部门
   - 姓名
   - 有效期（1年/2年/3年）

- 邮箱

5. **生成数字证书**

   X.509是密码学里公钥证书的格式标准。X.509证书已应用在包括TLS/SSL在内的众多网络协议里，同时它也用在很多非在线应用场景里，比如电子签名服务。X.509证书里含有公钥、身份信息（比如网络主机名，组织的名称或个体名称等）和签名信息（可以是证书签发机构CA的签名，也可以是自签名）。对于一份经由可信的证书签发机构签名或者可以通过其它方式验证的证书，证书的拥有者就可以用证书及相应的私钥来创建安全的通信，对文档进行数字签名。 另外除了证书本身功能，X.509还附带了证书吊销列表和用于从最终对证书进行签名的证书签发机构直到最终可信点为止的证书合法性验证算法。 X.509是ITU-T标准化部门基于他们之前的ASN.1定义的一套证书标准。证书组成结构标准用ASN.1（一种标准的语言）来进行描述. X.509 v3 数字证书结构如下：

   - 证书
     - 版本号
     - 序列号
     - 签名算法
     - 颁发者
     - 证书有效期
       - 此日期前无效
       - 此日期后无效
     - 主题
     - 主题公钥信息
       - 公钥算法
       - 主题公钥
     - 颁发者唯一身份信息
     - 主题唯一身份信息
     - ……
   - 证书签名算法
   - 数字签名
   - ----引用维基百科

6. **数据库存储证书**

7. **下载证书**

   - 下载证书页面非常简单，只需要输入要下载的证书序列号，就可以从浏览器获取相应的证书文件。

8. **接收用户的数字证书查询和注销**

   查询证书页面包括一个container，其中有用户查询证书需要提交的表单。该表单包括

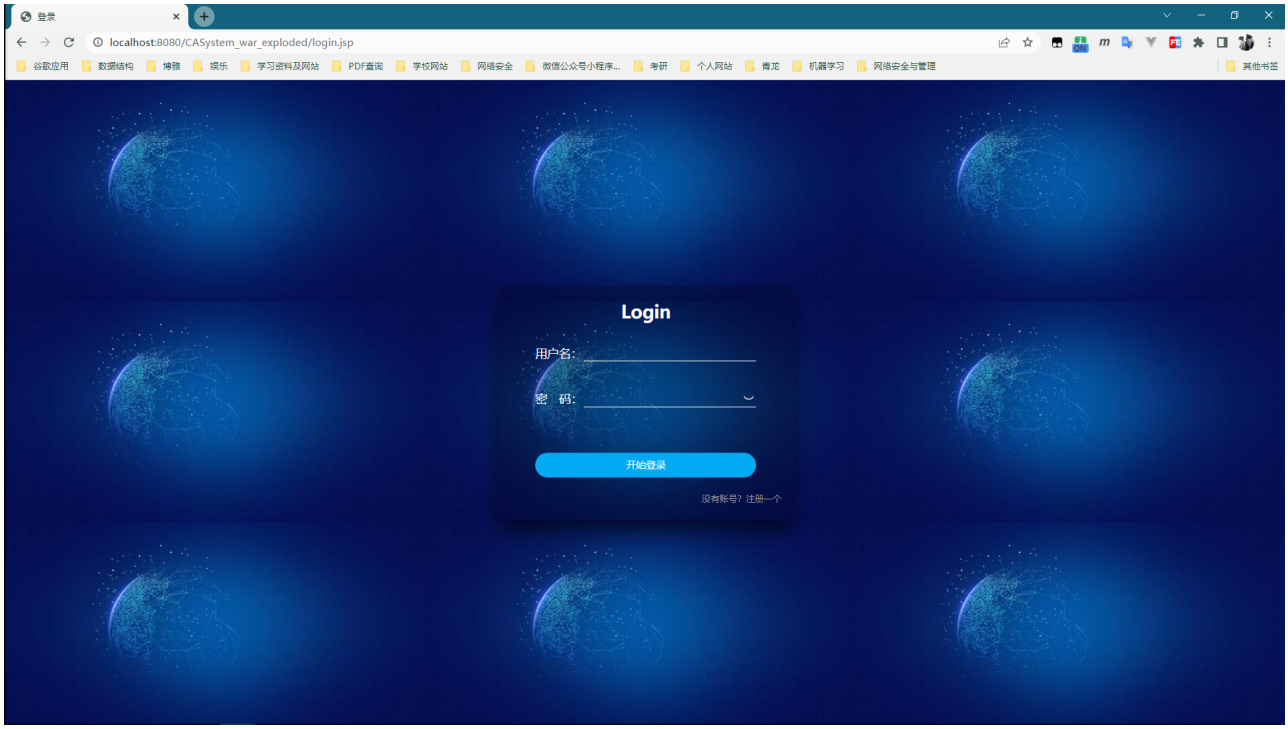
   - 证书申请者的用户名 端展示提供了证书的序列号、姓名、组织、证书有效期、证书有效期止信息，并提供了一个下载链接，单机该链接可以直接获取到该证书文件。

   注销证书页面包括一个container，其中有用户注销证书需要提交的表单。该表单包括

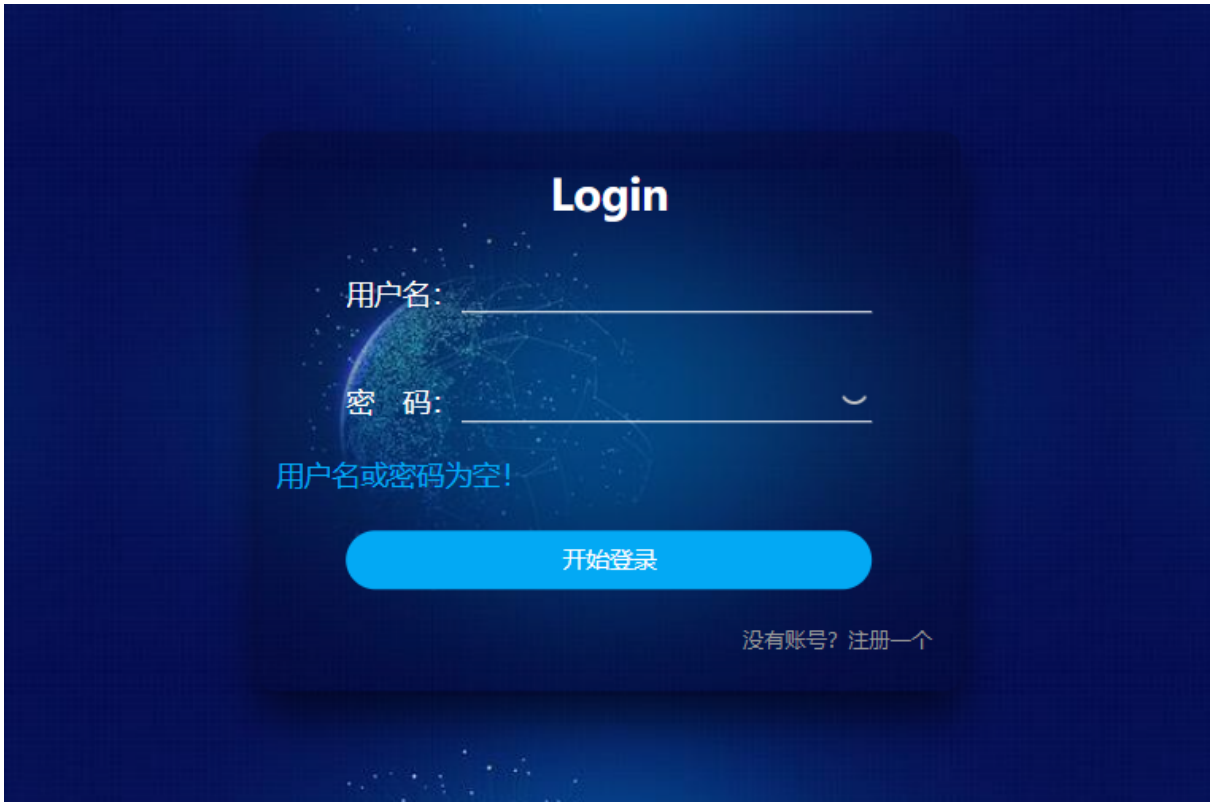
   - 证书申请者的用户名
   - 撤销证书页面也非常简单，只需要输入要下载的证书序列号，并且输入相应的撤销身份验证密码，就可以成功撤销该证书。证书被撤销后不会从证书库中删除，但会加入到 CRL 库中。
   - 撤销身份验证密码指的是证书在申请时，正在操作的用户的登录密码。这样设计虽然比较简单，但是限制了撤销权限，将撤销功能与用户登录状态相关联，由于证书序列号是（对所有人）可见的，所以限制请求者只有知道证书申请时的登录密码，才可以撤销成功。

四、实验步骤

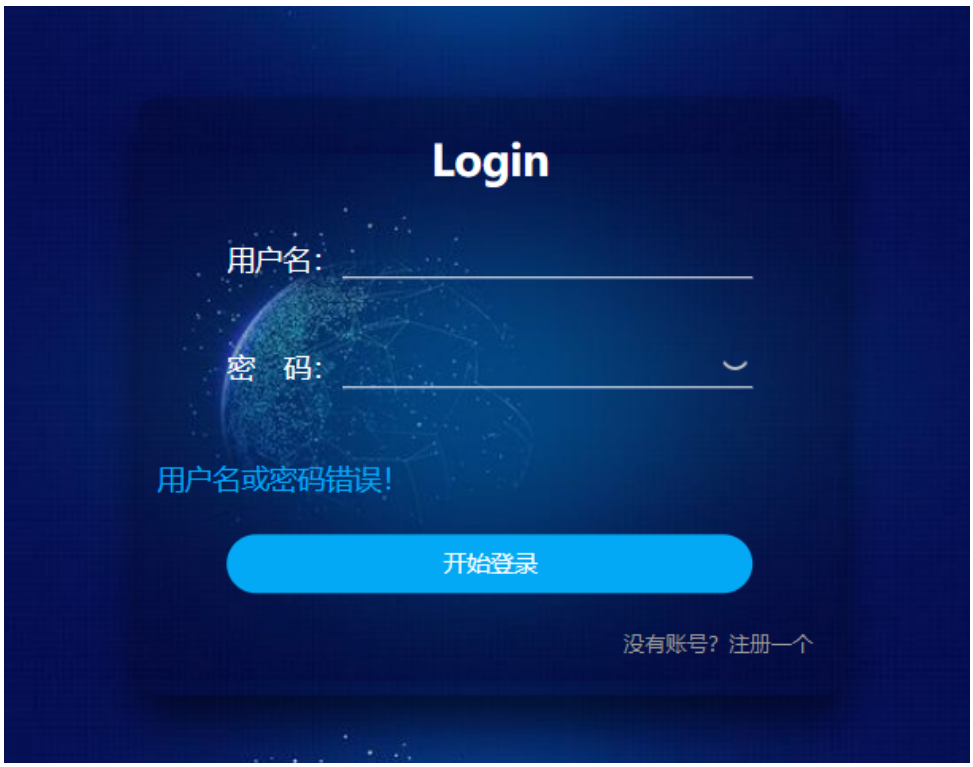
1. **注册与登录**



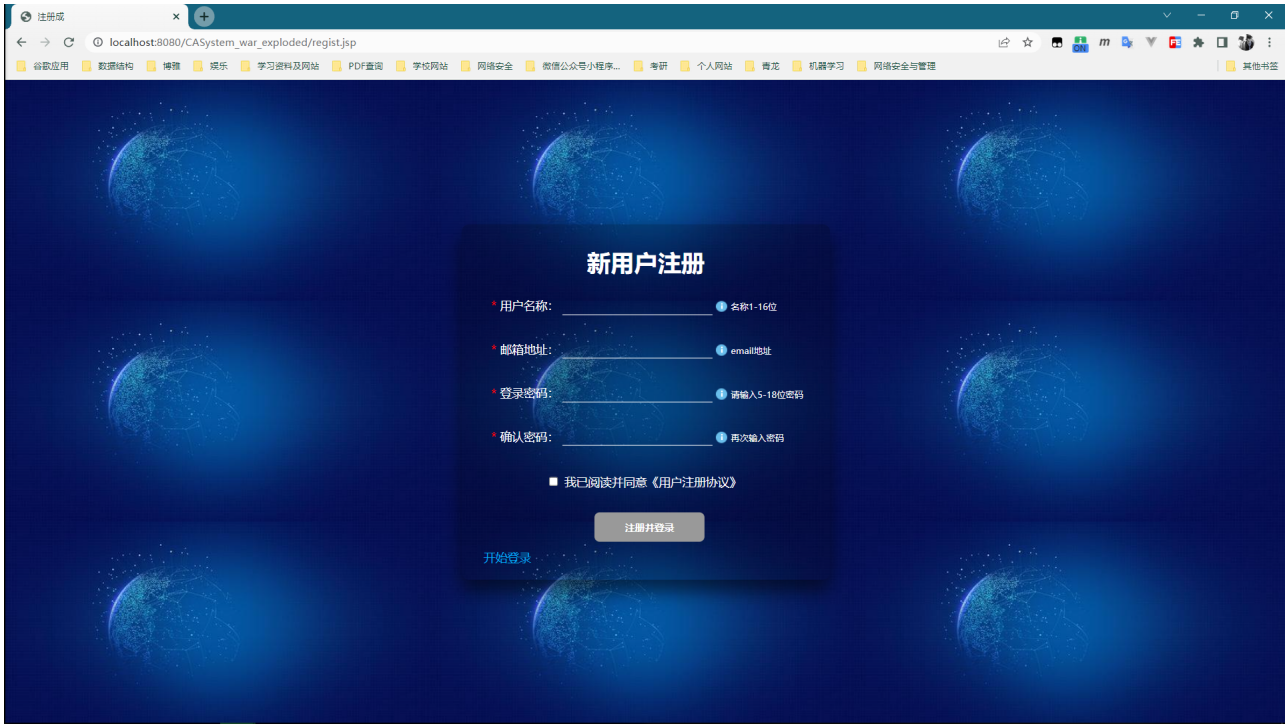
通过JavaScript验证对输入的用户名与密码进行简单核验，密码再通过md5加密，传输给后端，后端调用数据库进行用户名与密码查询，若查询的数据一致，则登录，否则无法登录。



用户名或密码为空



用户名或密码错误



通过JavaScript验证对输入的用户名，密码，电子邮箱进行简单核验，密码再通过md5加密，传输给后端，后端调用数据库对其信息进行存储。

登录核心代码

```
@WebServlet("/loginServlet")
public class LoginServlet extends HttpServlet {
    DbUtil dbUtil = new DbUtil();
    LoginDao loginDao = new LoginDao();
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.setCharacterEncoding("utf-8");
        //设置返回类型为json，字符集为utf-8
        response.addHeader("content-type", "text/json;charset=utf-8");
```

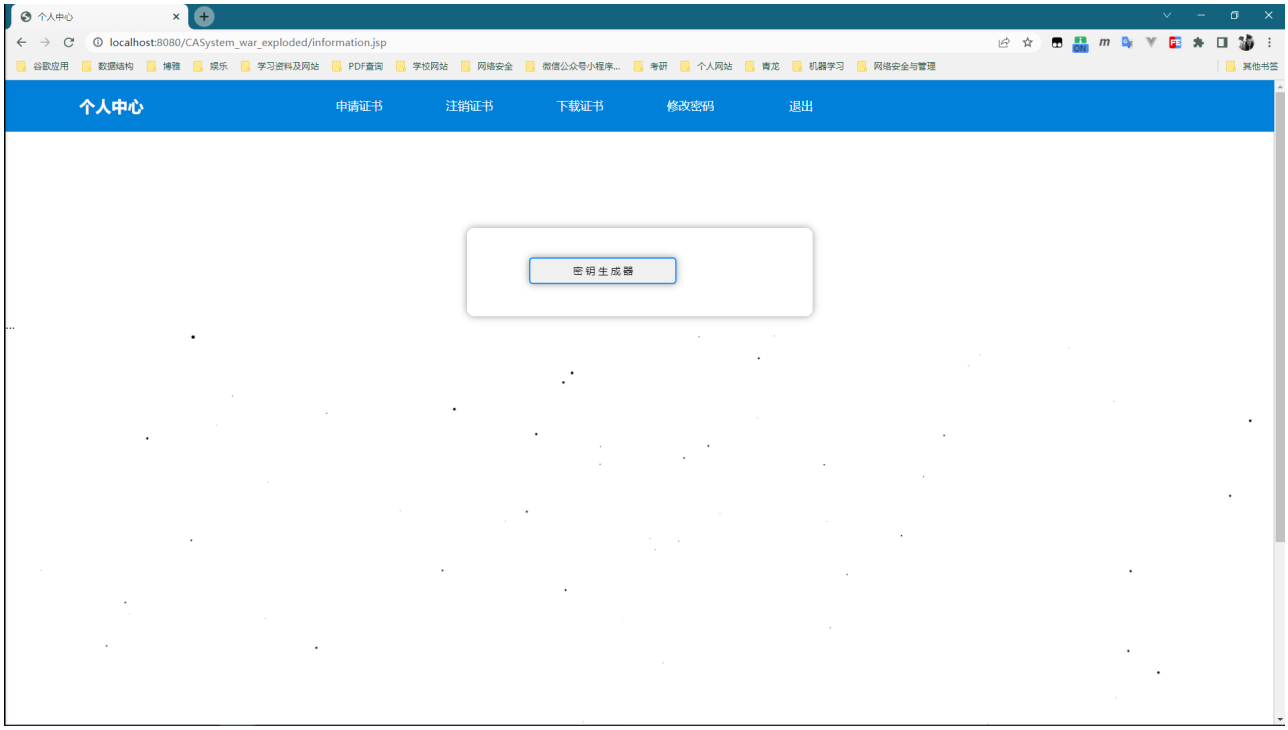
```
        HttpSession session = request.getSession();
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        System.out.println(username);
        System.out.println("joker468=" + password);
        if (StringUtil.isEmpty(username) || StringUtil.isEmpty(password)) {
            request.setAttribute("error", "用户名或密码为空！");
            request.getRequestDispatcher("login.jsp").forward(request, response);
            return;
        }
        User user = new User(username, password);
        Connection connection = null;
        try {
            connection = dbUtil.getConnection();
            User queryUser = loginDao.login(connection, user);
            if (queryUser == null) {
            request.setAttribute("LoginError", "用户名或密码错误！");
            // 服务器跳转
            request.getRequestDispatcher("login.jsp").forward(request, response);
            } else {
                session.setAttribute("currentUser", queryUser);
                session.setAttribute("username", username);
                request.getRequestDispatcher("information.jsp").forward(request, response);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

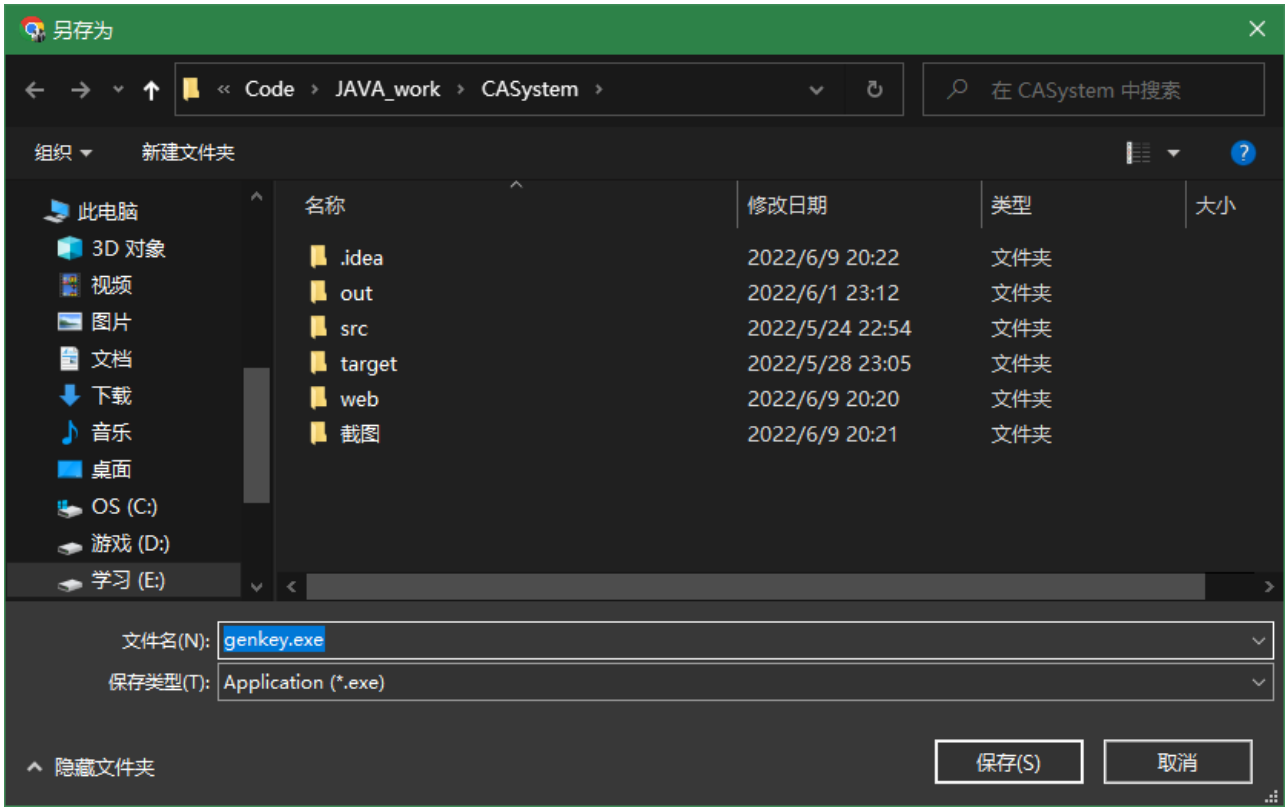
注册核心代码

```
@WebServlet(name = "RegistServlet", value = "/RegistServlet")
public class RegistServlet extends HttpServlet {
    DbUtil dbUtil = new DbUtil();
    RegistDao registDao = new RegistDao();
    Md5Util md5Util = new Md5Util();
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        this.doPost(request, response);
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws IOException {
        request.setCharacterEncoding("utf-8");
        //设置返回类型为json，字符集为utf-8
        response.addHeader("content-type", "text/json;charset=utf-8");
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        password = md5Util.getMd5ofStr(password);
        System.out.println(password);
        String email = request.getParameter("email");
        String created_at = DateUtil.formatDate(new Date(), "yyyy-MM-dd HH:mm:ss");
        String update_at = DateUtil.formatDate(new Date(), "yyyy-MM-dd HH:mm:ss");
        String deleted_at = DateUtil.formatDate(new Date(), "yyyy-MM-dd HH:mm:ss");
        User user = new User(created_at, update_at, deleted_at, username, password, email, "0");
        Connection connection;
        try {
            connection = dbUtil.getConnection();
            if (registDao.regist(connection, user)) {
            out.println("添加成功，三秒后返回登陆頁面");
            response.setHeader("refresh", "3;url= login.jsp");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
```

```
        }
    }
```

2. **申请公钥**





点击密钥生成器后，马上从浏览器获取一个genkey.exe文件，运行该可执行程序将在与该文件所在路径同级的目录下生成两个文件pk.key和sk.key。前者为公钥文件，后者为私钥文件，两者共同组成一对 1024bits 的 RSA 公私密钥对。

核心代码

```
@WebServlet(name = "DownloadServlet", value = "/DownloadServlet")
public class DownloadServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws IOException {
```

```
        String filename = request.getParameter("filename");
        System.out.println(filename);
        ServletContext servletContext = this.getServletContext();
        String file_path = servletContext.getRealPath("download/" + filename);
        String mimeType = servletContext.getMimeType(file_path);
        System.out.println(file_path);
        response.setHeader("content-type", mimeType);
        response.setHeader("content-disposition", "attachment;filename=" + filename);
        FileInputStream fileInputStream = new FileInputStream(file_path);
        ServletOutputStream servletOutputStream = response.getOutputStream();
        byte[] buff = new byte[1024 * 8];
        int len;
        while ((len = fileInputStream.read(buff)) != -1) {
            servletOutputStream.write(buff, 0, len);
        }
        fileInputStream.close();
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        this.doPost(request, response);
    }
}
```

3. **申请证书**



用户为其输入各种信息，信息加密传输到后台，与公钥配合进行解密，生成数字证书，并返回一个证书序列号，用户可以根据该证书序列号进行下载或者注销。

4. **注销证书**



撤销证书页面也非常简单，只需要输入要下载的证书序列号，并且输入相应的撤销身份验证密码，就可以成功撤销该证书。证书被撤销后不会从证书库中删除，但会加入到 CRL 库中。

撤销身份验证密码指的是证书在申请时，正在操作的用户的登录密码。这样设计虽然比较简单，但是限制了撤销权限，将撤销功能与用户登录状态相关联，由于证书序列号是（对所有人）可见的，所以限制请求者只有知道证书申请时的登录密码，才可以撤销成功。

注销证书核心代码

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    request.setCharacterEncoding("utf-8");
    CertificateDao certificateDao = new CertificateDao();
    String serial_number = request.getParameter("serial_number");
    String revoke_pwd = request.getParameter("revoke_pwd");
    String sign_serial_number = request.getParameter("sign_serial_number");
    String sign_revoke_pwd = request.getParameter("sign_revoke_pwd");
    PrivateKey privateKey;
    try {
        privateKey =
                Ende.loadPrivateKeyByStr("MIICeQIBADANBgkqhkiG9w0BAQEFAASCAmMwggJfAgEA" +
                        "AoGBAKIRy1Ayqp5COdLW9ToFc5pVOGFqdRw5PruPOLDbk25VQoQeNmCzX3g47PkhnTD1" +
                        "Qa3Ofn2ikHDLfb3lQpv2vZAsxdMVaHzzN5nsoXgss16ydiHDLQwheSdvqXGcnsN+fPk" +
                        "r6+AYl3Q7QuZNQkND7GeWCmSHvHaNWsMmCVxTkzRpAgMBAAECgYEAimdFyEwsdpA5z" +
                        "zsxGoaTTaYfStnd/udIEmZh1G7/fYakEi225GfqTMHYZXz2P1wC5cnlLadJUHoG/M" +
                        "cvVf+lq3+6ph2pA6wHI4bvMSPS9SoLkcLNiggcqAKnySu2X9ZNCEza+NGMj726Ixs" +
                        "yxB7+3PCzSfiiwzv3JvxkyA614AECQQDen/yaEpO0F+mh4aQe6e9pj2xjJuLb7so" +
                        "oDDFOJaotxnY/1aaNdongbqA2j7HeBf6isv19F+w+nWKWXLVoOTQBAkEAul3Hsm" +
                        "FH5z8fwiv+B9AGrK6vfbadBqlTeK1gMxqDPDUuDMNbW4rPRlCXEBaoUMSFRDFNM" +
                        "rfFPejnn5tu1pzgaQJBANvj8kDMcI/FvsJieRT/w7XkMA6PbiwF5C9CO8EQetLT" +
                        "4CCVCvlXSEAhhKXfsLO4ABb77F0OsA34rlQOJjBXsAECQQCS+cS07D2NpN3B/3n" +
                        "O5YNuCjICfdMm3sEiqfD1PJKFGBeiHytcbYN8G7CXEpdZYzMKjaspNX8LjTOmTy" +
                        "nBfWUJAkEAtWOybjlu0zLSiC3gTTyK1Q+aS1eLji/a9l/XdG7L9dQ0fEpfB8XTt" +
                        "UQdmmt3vDAe3qwyV5t/vJkfZOrHY+yIYg==");
        serial_number =
                new String(Ende.decrypt((RSAPrivateKey) privateKey
                        , Base64.getDecoder().decode(serial_number)), "utf-8");
        revoke_pwd =
                new String(Ende.decrypt((RSAPrivateKey) privateKey
                        , Base64.getDecoder().decode(revoke_pwd)), "utf-8");
    } catch (Exception e) {
        e.printStackTrace();
    }
    if (!SHADigest.getDigest(serial_number).equalsIgnoreCase(sign_serial_number)
            || !SHADigest.getDigest(revoke_pwd).equalsIgnoreCase(sign_revoke_pwd)) {
```

```
            request.setAttribute("msg", "消息已损坏！请报警！");
            request.getRequestDispatcher("revoke_cer.jsp").forward(request, response);
            return;
        }
        String username = request.getSession().getAttribute("username").toString();
        LoginDao userDao = new LoginDao();
        User presentUser = new User();
        DbUtil dbUtil = new DbUtil();
        Connection connection = null;
        try {
            connection = dbUtil.getConnection();
        } catch (Exception e) {
            e.printStackTrace();
        }
        presentUser.setUsername(username);
        presentUser.setPassword(revoke_pwd);
        try {
            if (userDao.login(connection, presentUser) != null) {
                request.setAttribute("msg", "撤销身份验证密码错误！请您确认登录信息后输入正确的验证密码！");
                request.getRequestDispatcher("cancel_the_certificate.jsp").forward(request, response);
                return;
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        String[] revoke_msg = certificateDao.revoke(serial_number);
        if (revoke_msg != null) {
            request.setAttribute("msg", "此证书已被成功撤销！");
            List<Mycrl> crlList = certificateDao.getCrl();
            ServletContext servletContext = this.getServletContext();
            String file_path = servletContext.getRealPath("/download");
            File file = new File(file_path);
            if (!file.exists()) {
                file.mkdir();
            }
            String file_name = file_path + "/" + "crl.xml";
            file = new File(file_name);
            FileWriter fileWriter = new FileWriter(file);
            BufferedWriter bufferedWriter = new BufferedWriter(fileWriter);
            bufferedWriter.write("<?xml version='1.0' encoding='gbk'?>");
            bufferedWriter.write("<crls>\n");
            for (Mycrl mycrl : crlList) {
                bufferedWriter.write("<crl>\n");
                bufferedWriter.write(
                        "<serial_number>" + mycrl.getSerial_number() + "</serial_number>\n");
                bufferedWriter.write(
                        "<organization>" + mycrl.getOrganization() + "</organization" + ">\n");
                bufferedWriter.write("<start_time>" + mycrl.getStart_time() + "</start_time>\n");
                bufferedWriter.write("<end_time>" + mycrl.getEnd_time() + "</end_time>\n");
                bufferedWriter.write("</crl>\n");
                bufferedWriter.flush();
            }
            bufferedWriter.write("</crls>\n");
            bufferedWriter.close();
            fileWriter.close();
            request.setAttribute("serial_number", serial_number);
            request.setAttribute("organization", revoke_msg[1]);
            request.getRequestDispatcher("revoke_result.jsp").forward(request, response);
        } else {
            request.setAttribute("msg", "此证书不存在或已失效！");
            request.getRequestDispatcher("revoke_cer.jsp").forward(request, response);
            return;
        }
    }```
```

5. **下载证书**





通过前面获取的证书序列号与登陆密码进行证书下载

下载证书核心代码

```
@WebServlet(name = "DownloadCerServlet", value = "/DownloadCerServlet")
 public class DownloadCerServlet extends HttpServlet {

 @Override
 protected void doPost(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
     String serial_number = request.getParameter("serial_number");
     System.out.println(serial_number);
     CertificateDao certificateDao = new CertificateDao();
     String filePath = certificateDao.getFilePath(serial_number);
     if (filePath == null) {
         request.setAttribute("msg", "此证书不存在！");
         request.getRequestDispatcher("download.jsp").forward(request, response);
         return;
     }
     String filename = serial_number + ".mycer";
     ServletContext servletContext = this.getServletContext();
     //        String mimeType = servletContext.getMimeType(file_path);
     response.setHeader("content-type", "application/octet-stream");
     response.setHeader("content-disposition", "attachment;filename=" + filename);
     FileInputStream fileInputStream = new FileInputStream(filePath);
     ServletOutputStream servletOutputStream = response.getOutputStream();
     byte[] buff = new byte[1024 * 8];
     int len = 0;
     while ((len = fileInputStream.read(buff)) != -1) {
         servletOutputStream.write(buff, 0, len);
     }
     fileInputStream.close();
 }

 @Override
 protected void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
     this.doPost(request, response);
 }
}
```

6. **查询证书**



查询证书核心代码

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
    CertificateDao certificateDao = new CertificateDao();
    int everyPage = 1;
    int totalCount = certificateDao.getCount();
    String scurrentPage = request.getParameter("currentPage");
    int currentPage = 1;
    if (scurrentPage == null) {
        currentPage = 1;
    } else {
        currentPage = Integer.parseInt(scurrentPage);
    }
    Page page = PageUtil.createPage(everyPage, totalCount, currentPage);
    List<CertItem> certItems = certificateDao.getCerByPage(page);
    System.out.println(page.toString());
    request.setAttribute("page", page);
    request.setAttribute("certItems", certItems);
    request.getRequestDispatcher("information.jsp").forward(request, response);
}```
```

7. 证书设计

X.509是密码学里公钥证书的格式标准。X.509证书已应用在包括TLS/SSL在内的众多网络协议里，同时它也用在很多非在线应用场景里，比如电子签名服务。X.509证书里含有公钥、身份信息（比如网络主机名，组织的名称或个体名称等）和签名信息（可以是证书签发机构CA的签名，也可以是自签名）。对于一份经由可信的证书签发机构签名或者可以通过其它方式验证的证书，证书的拥有者就可以用证书及相应的私钥来创建安全的通信，对文档进行数字签名。 另外除了证书本身功能，X.509还附带了证书吊销列表和用于从最终对证书进行签名的证书签发机构直到最终可信点为止的证书合法性验证算法。 X.509是ITU-T标准化部门基于他们之前的ASN.1定义的一套证书标准。

- 参考X.509证书结构，设计自己的证书结构如下：
- 序列号（Serial Number: ）——以当前时间为前缀的组织机构名称（如，20191222091200HIT）的16进制md5散列值。
- 签名算法(Sign Algorithm:)——sha1RSA
- 签名哈希算法(Encrypt Algorithm:)——sha256
- 颁发者——GothamCityTrust RSA CA 2019, www.tofushen.com, Gotham City Trust, CN
- 有效期从(Valid Time From:)——当前日期，如2019年12月21日 21:32:00
- 到(Valid Time To:)——当前日期向后（1年/2年/3年），如2022年12月21日 21:32:00
- 使用者(User:)——组织机构名

- 公钥(Public Key:)——公钥文件字串
- 签名(Sign:)——公钥文件字串的签名值，使用sha1RSA算法哈希，RSA算法加密，CA私钥加密。
- 证书文件内容为文本形式，文件格式为.mycer。
- 因为证书的申请要和登录者信息相关联，所以在数据库中存储的证书申请信息如下：
  - 用户名
  - 序列号
  - 组织
  - 文件路径（用于下载证书）
  - 有效期起
  - 有效期止
  - 法人姓名
  - 经办人姓名
  - 经办人邮箱

五、总结

本次网络安全与管理课程设计，加强了自己的动手能力，从最开始的选题，到构思设计框架，再到后期的一步一步实现功能，逐渐完成具有下载，申请，注销，查询等具有基本功能的CA系统。

此次课程设计当中，遇到的最大的困难就是加密传输信息，对于公钥与私钥的结构不熟悉，RSA消息加密传输的一些不理解。

在完成此次的课程设计后，对这些问题有了基本的理解，能基本了解他们的的工作原理。

六、附录

1. https://zh.wikipedia.org/wiki/X.509 “X.509，维基百科”
2. https://blog.csdn.net/wangliang369/article/details/83792116 “jsp 实现分页操作”
3. https://blog.csdn.net/ayang1986/article/details/80810072 “CA认证简单介绍与工作流程”
4. https://www.cnblogs.com/demodashi/p/8458113.html “Java使用RSA加密解密签名及校验”
5. https://www.jianshu.com/p/c65fa3af1c01 “PKI/CA工作原理及架
6. https://blog.csdn.net/ayang1986/article/details/80810072 “CA认证简单介绍与工作流程”
7. https://www.cnblogs.com/raphael1982/p/8012634.html “用户名、密码等15个常用的js正则表达式”